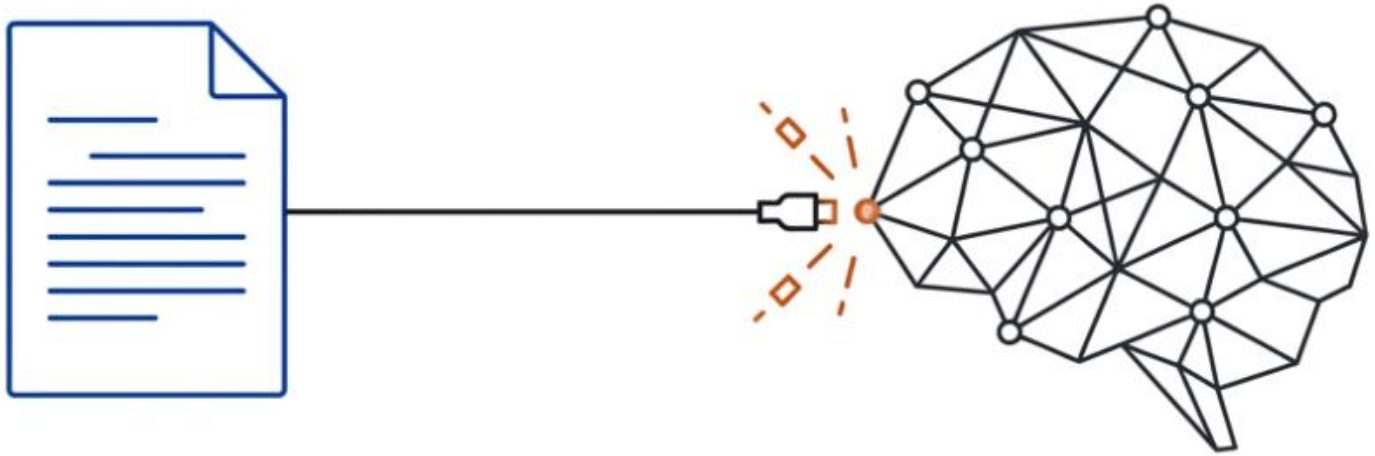


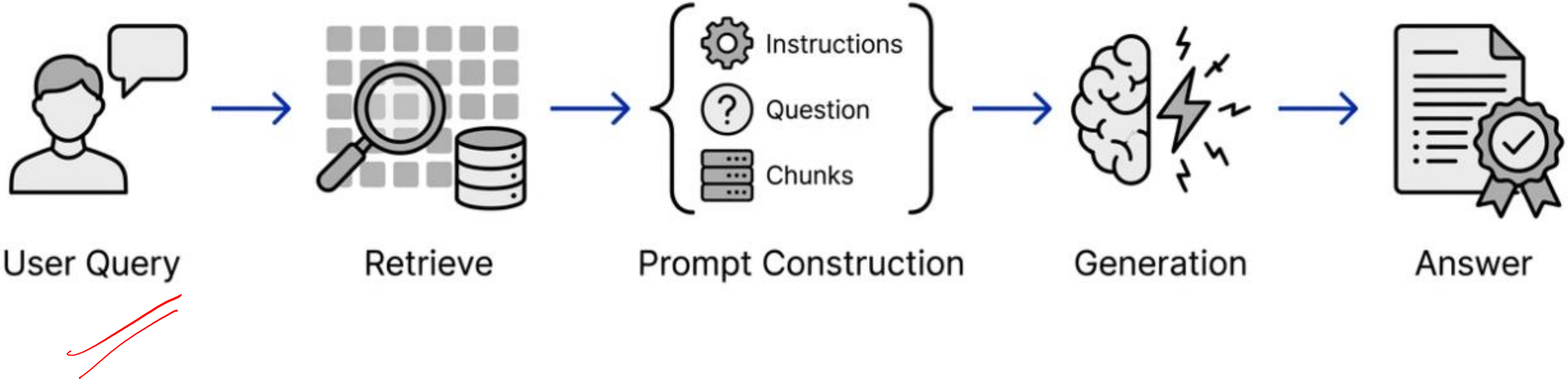
Search Foundations

Vector Embeddings - Sparse and Dense and Hybrid, Scoring.
Metrics: Precision@K, Recall@K, MRR

Sakharam Gawade
AI Productization / Platform Intelligence Team

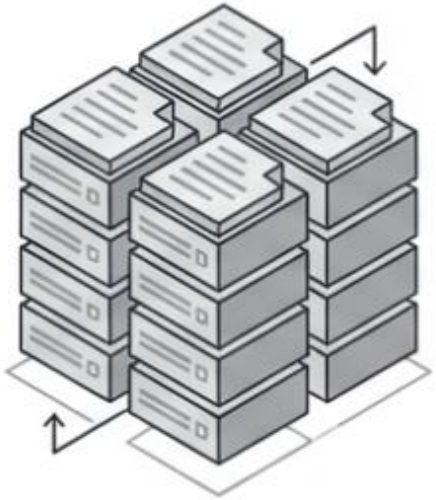
https://tinyurl.com/rag-augmented

The End-to-End RAG Flow



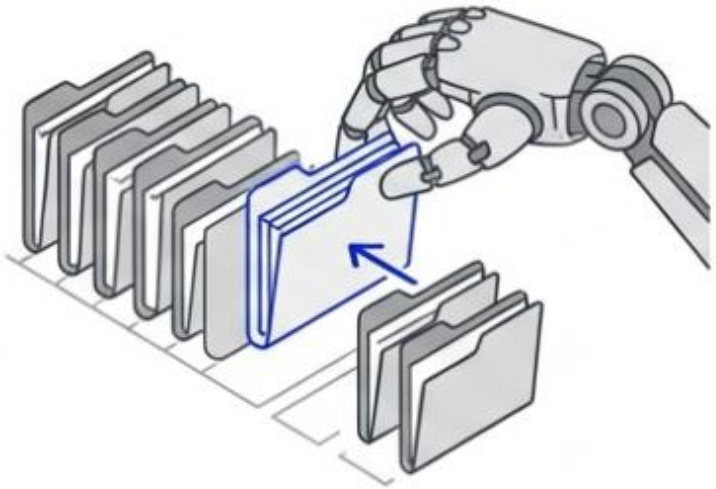
This flow turns a creative writing machine into a grounded research assistant.

The Store
(The Library)



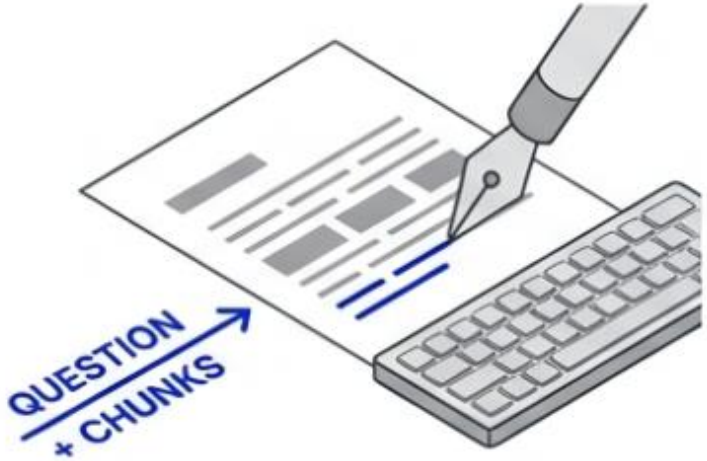
Holds your documents in a retrieval-friendly form.

The Retriever
(The Librarian)



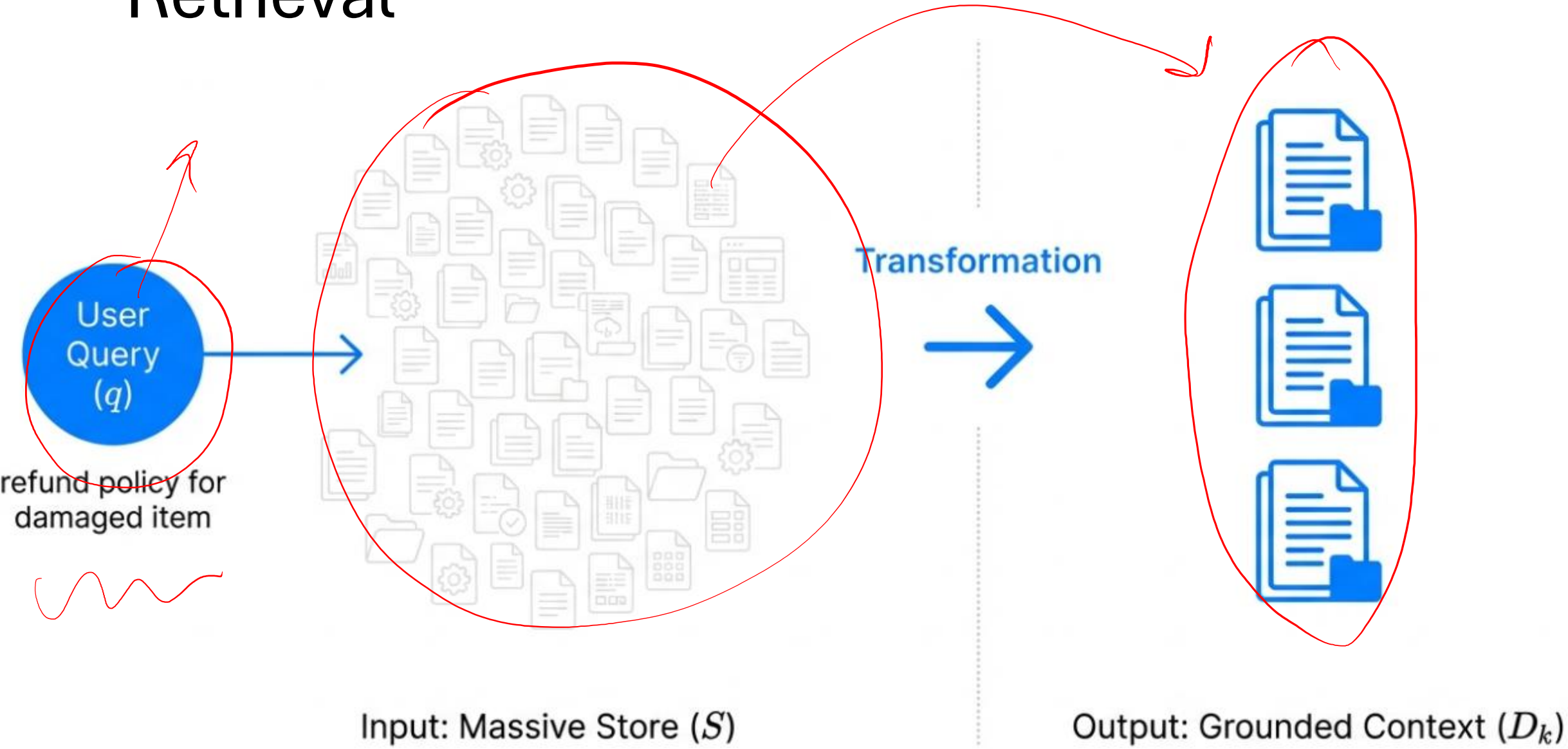
Fetches the top relevant chunks based on a query.

The Generator
(The Writer)



Reads the question + chunks to synthesize an answer.

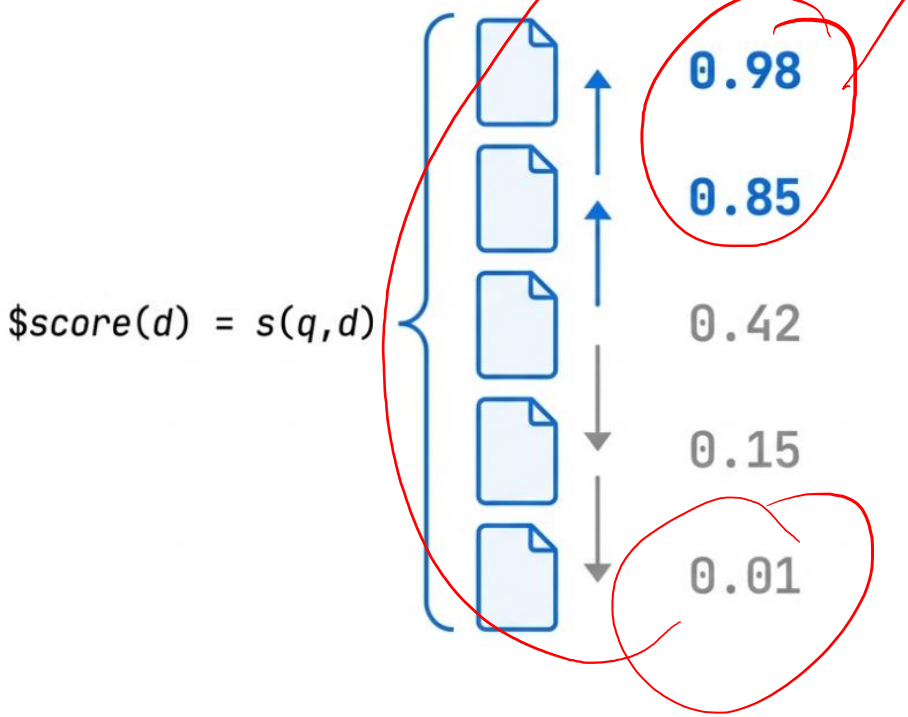
Retrieval


User Query (q)
refund policy for damaged item
Transformation
Input: Massive Store (S)
Output: Grounded Context (D_k)

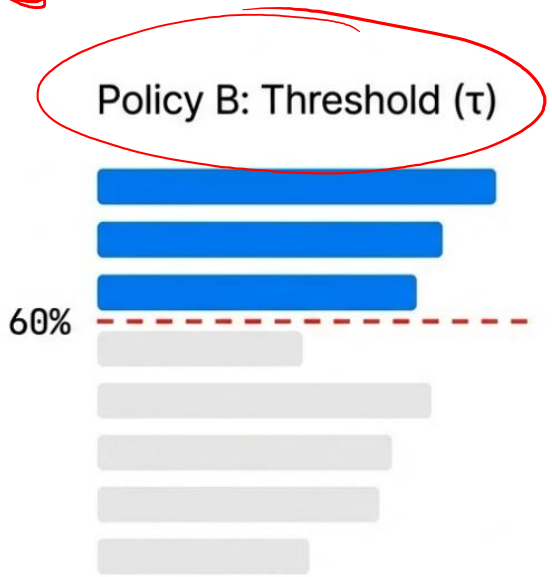
Scoring Relevant Documents

0 – 1

0.7

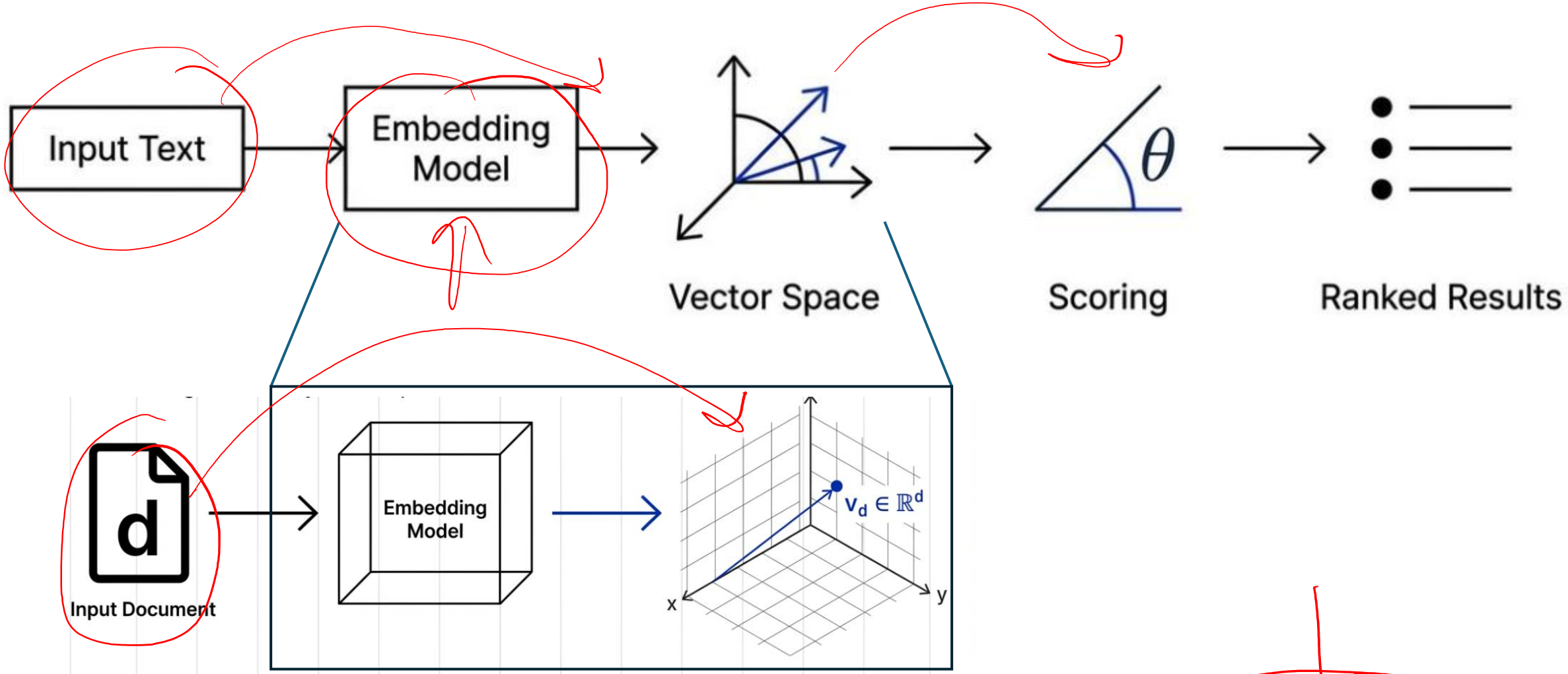


Controls Budget (Fixed Count)



Controls Precision (Minimum Confidence)

Representing Text (Document and Query)



Retrieval shifts from matching keywords to finding the most similar vector in space.

https://projector.tensorflow.org/

Scoring: Dot Product

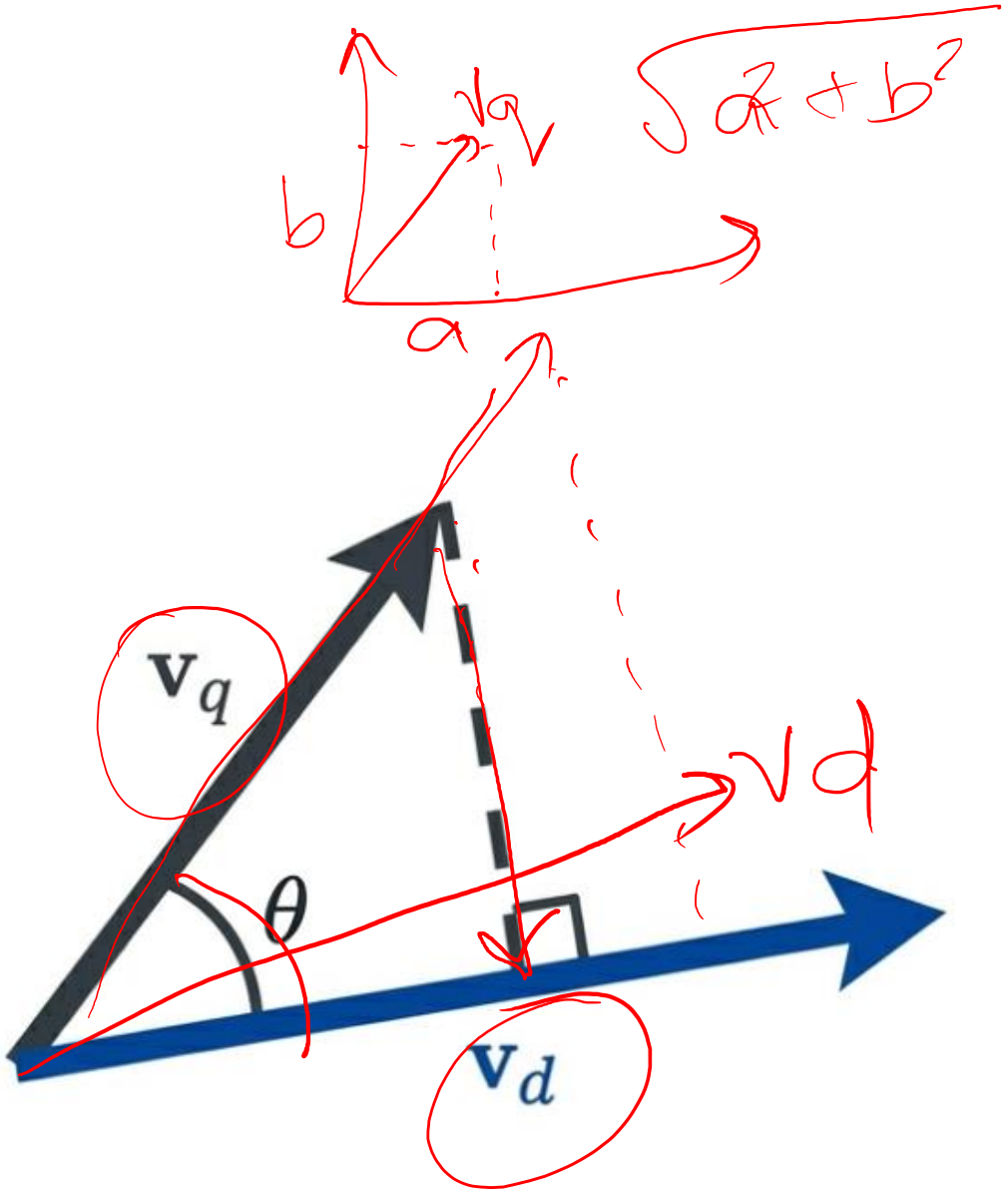
$$\mathbf{v}_q \cdot \mathbf{v}_d = |\mathbf{v}_q||\mathbf{v}_d|\cos(\theta)$$

Mechanics:

- Score increases with Alignment (small θ)
- Score increases with Magnitude (large |v|)

Implication:

Relevant docs align with the query, but high confidence/importance (magnitude) boosts the score.



Scoring: Cosine Similarity

$[1/\sqrt{2} \;\; 1/\sqrt{2}] \begin{bmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{bmatrix} = 1$

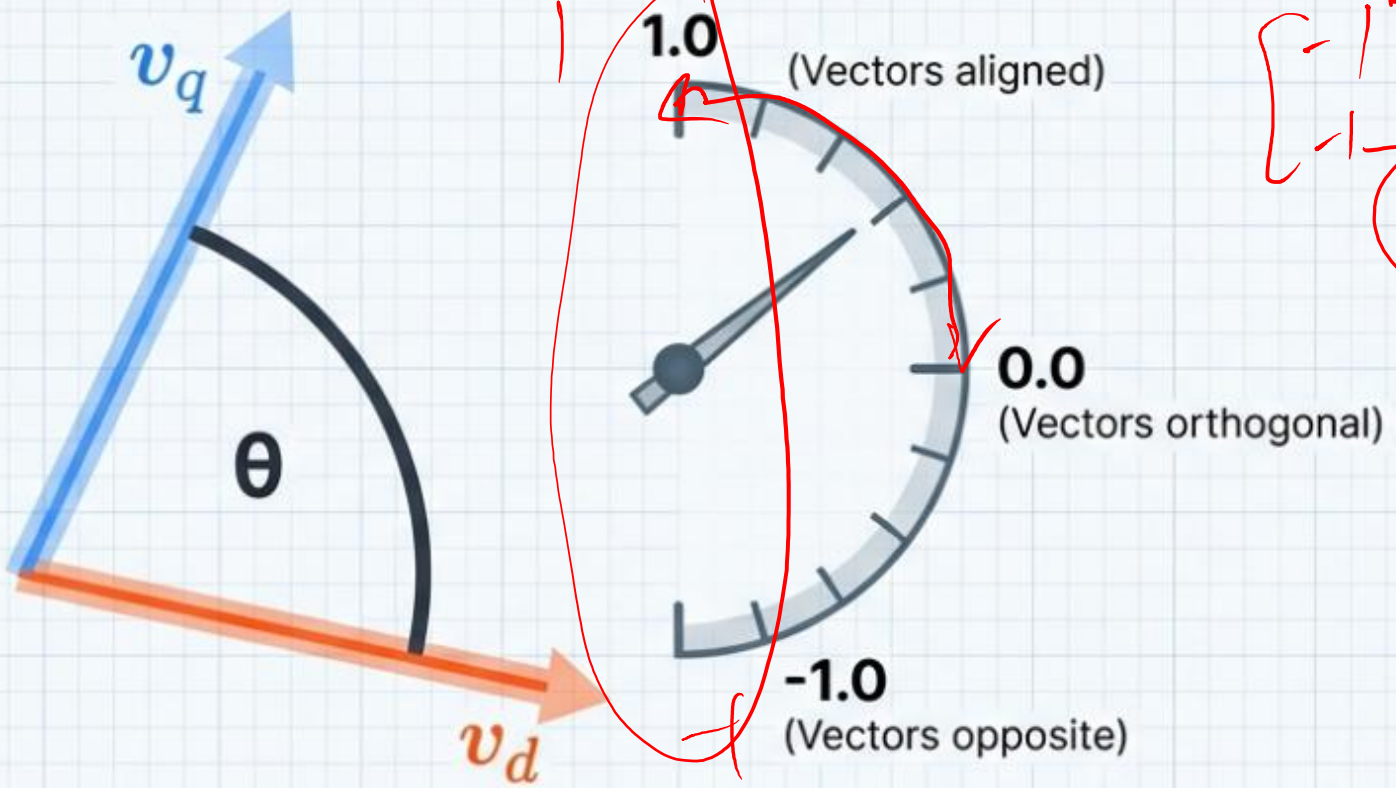
$\sqrt{1^2+1^2}$

$\sqrt{2}$

$[1]_q / \sqrt{2}$

$[-1]_d$

-1



$$\cos(\theta) = \frac{v_q \cdot v_d}{||v_q||v_d|}$$

THE NORMALIZER:
Divides by the product of magnitudes to isolate direction.

Lexical Search

Document 1 (Positive Match)

Laptop battery is not charging. Check the power adapter and the charging port.

Document 2 (Weak Match)

If the laptop will not turn on, connect the charger and try a hard reset.

Document 3 (Negative Match)

Refund policy: request a refund within 30 days for a damaged item.

User Query: `laptop battery not charging`

Step 1: Create a set of terms (vocab)

Input:

Laptop battery is not charging...

Apply Filter

Output Tokens:

Term Frequency > 1

[laptop, battery, not, **charging**, check, power, adapter, **charging**, port]

The complete universe of known terms (N=22).

V = { adapter, **battery**, charger, **charging**, check, connect, damaged, days, hard, item, **laptop**, not, on, policy, port, power, request, refund, reset, try, turn, within }

TF Matrix (Term Frequency)

$$tf(t,d) = f(t,d)$$

Term	D1	D2	D3
adapter	1	0	0
battery	1	0	0
charging	2	0	0
laptop	1	1	0
not	1	1	0
refund	0	0	2

tf(t,d) = count of term **t** in doc **d**

IDF: Common Words are poor signals

$$idf(t) = \log\left(\frac{N}{df(t)}\right)$$

Rare words are rewarded. Common words are penalized.

Common Terms

Terms appearing in 2/3 documents
(e.g., ‘laptop’, ‘not’)

```
idf = log(3 / 2) ≈ 0.4055
```

Low Weight

Rare Terms

Terms appearing in 1/3 documents
(e.g., ‘battery’, ‘charging’)

```
idf = log(3 / 1) ≈ 1.0986
```

High Weight

TF-IDF

$$w(t,d) = tf(t,d) \cdot idf(t)$$

Term: “charging” TF (2) × IDF (1.0986) = **2.1972**

Term: “battery” TF (1) × IDF (1.0986) = 1.0986

Term: “laptop” TF (1) × IDF (0.4055) = 0.4055

D1

$$\begin{bmatrix} 2.1972 \\ 1.0986 \\ 0.4055 \end{bmatrix}$$

Obtaining Similarity

1	**Document 1**		Score 0.7129 - **PASSED**
2	**Document 2**		Score 0.0670 - FAILED (Below Threshold)
3	**Document 3**		Score 0.0000 - FAILED

BM25

Term Importance Weight

$$\mathrm{BM25}(q,d) = \sum_{q} \mathrm{idf}(t) \cdot \left[\frac{\mathrm{tf}(t,d) \cdot (k1+1)}{\mathrm{tf}(t,d) + k1 \cdot \left(1 - b + b \cdot \frac{||d|}{\mathrm{avgdl}}\right)} \right]$$

Raw Frequency Count

Document Length Penalty

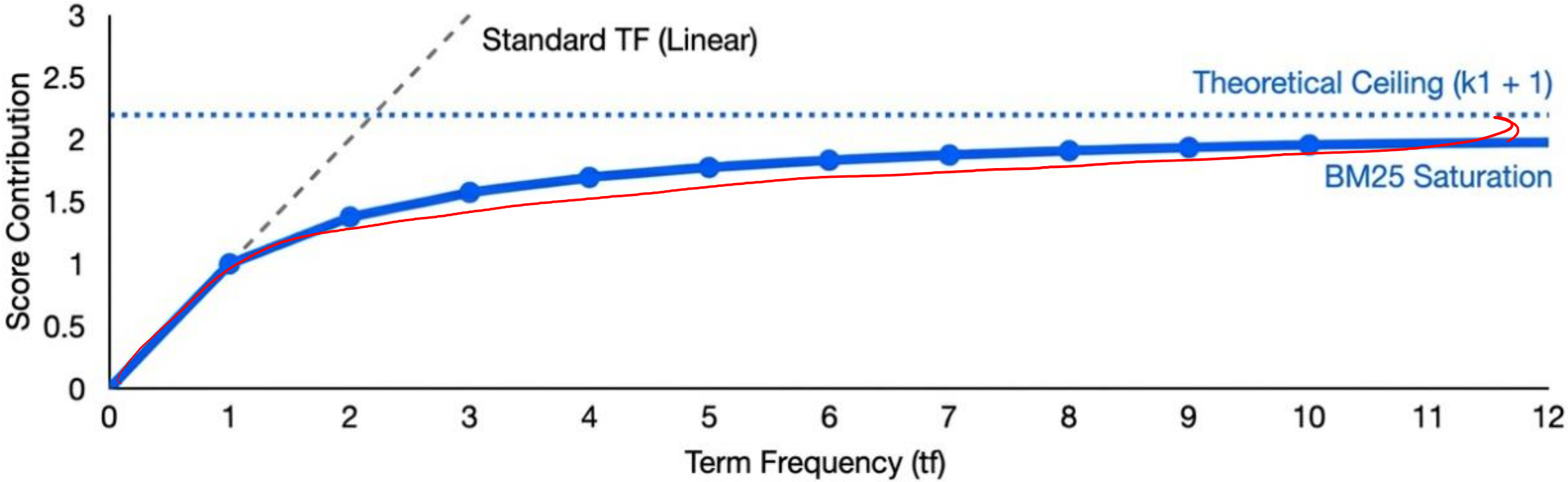
Parameters

Saturation (k1) = 1.2

Length Norm (b) = 0.75

TF Saturation – k1

Why repeating a word 100 times doesn't make it 100x relevant



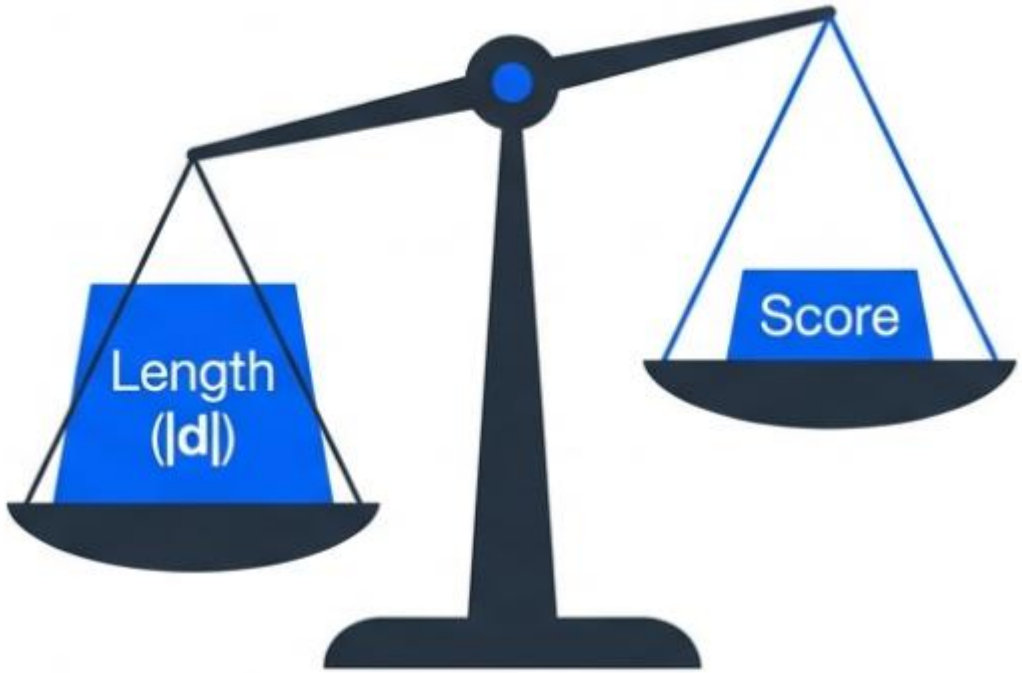
The k1 factor (1.2) enforces diminishing returns.

Length normalization

The Factor **K**:

$$K = k1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avg}dl}\right)$$

Interpretation: As document length |d| increases, **K** increases, reducing the final score.



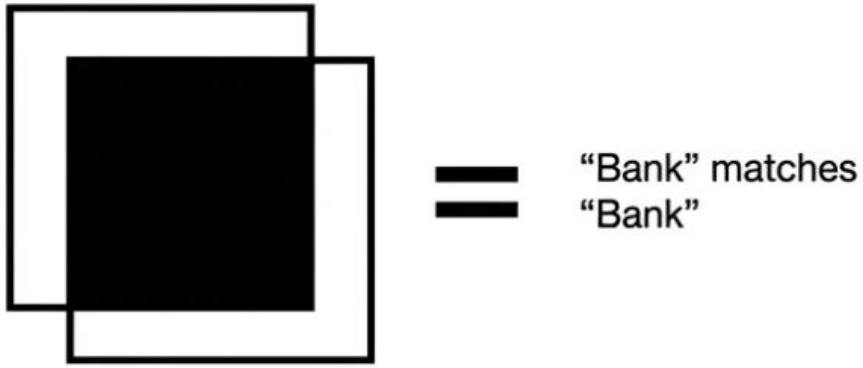
Average Length (**avgdl**) = 8.67

If |d| > 8.67, the score is penalized.
If |d| < 8.67, the score is boosted.

Semantic Search

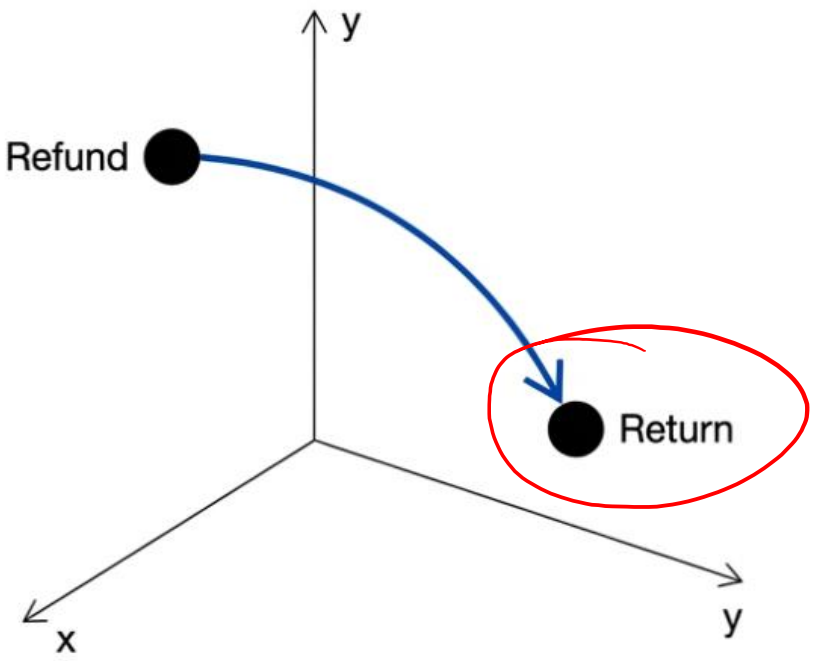
Matching Words vs. Matching Meaning

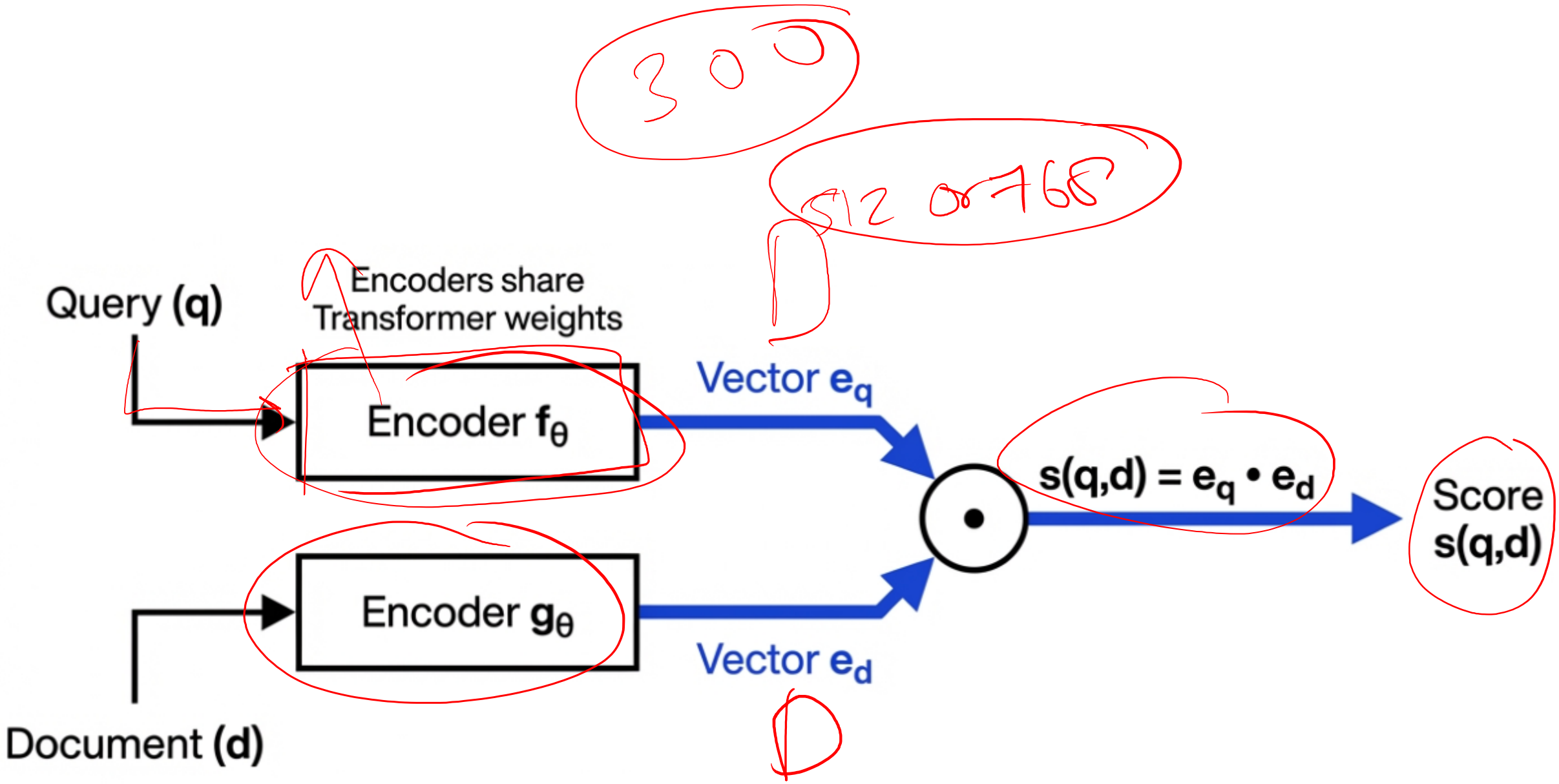
LEXICAL / TF-IDF



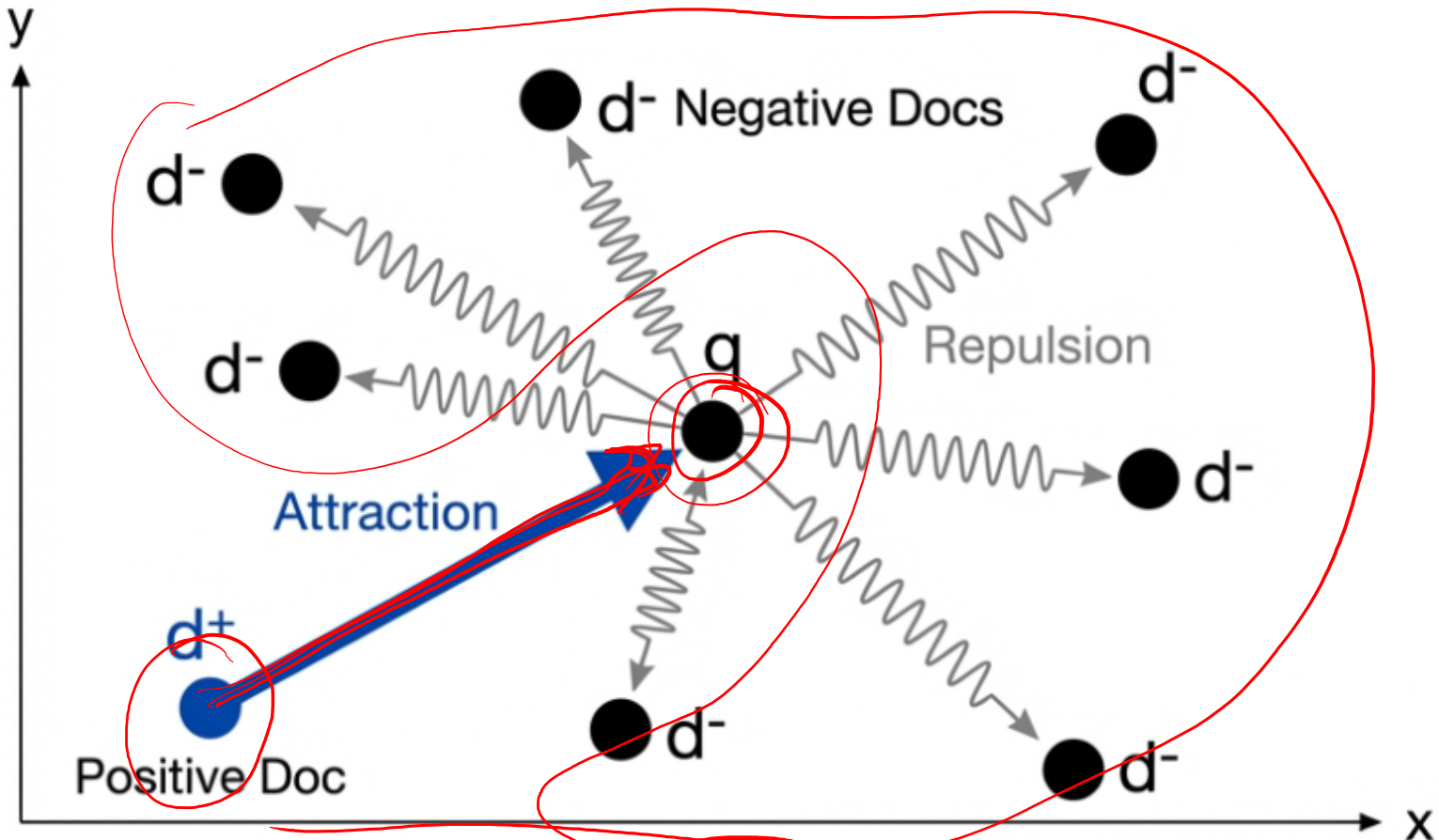
Lexical methods match tokens. Semantic similarity matches meaning by mapping text to a dense vector embedding, capturing relationships even when vocabulary differs.

SEMANTIC / DENSE




300
512 or 768
Query (q)
Encoders share
Transformer weights
Encoder f_θ
Vector e_q
$s(q,d) = e_q \bullet e_d$
Score
s(q,d)
Encoder g_θ
Vector e_d
Document (d)

Learning Objective



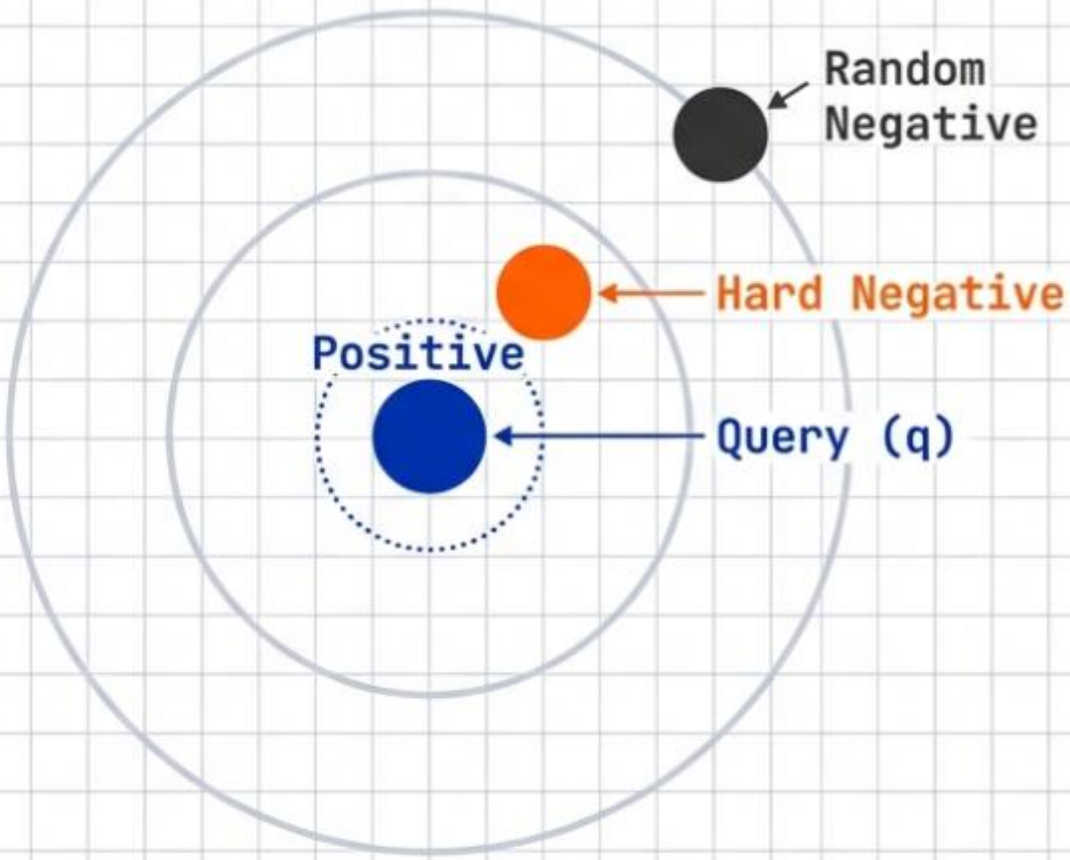
The model learns a geometric space where relevant pairs are close and irrelevant pairs are far.

$$L = -\log\left(\frac{\exp(s(q, d^+))}{\sum \exp(s(q, d^-))}\right)$$

$$\frac{0.9}{1.2}$$

Hard Negatives

Random negatives are too easily distinguished. Systems must mine “hard negatives”—documents that look similar but are factually wrong.



Hard negatives force the embedding model to refine its understanding of semantic nuance.

Paraphrase Equivalence	'Reset phone' ⟷ (approx equal) 'Factory reset steps'
Synonymy	'Refund' ⟷ (approx equal) 'Return'
Intent	Distinguishing 'Troubleshoot' vs. 'Buy'
Topical Relatedness	Battery ⟷ Charger ⟷ Adapter
Entity Semantics	Categorizing iPhone/Android as 'Devices' (→ Devices)

"TF-IDF represents words.
Dense embeddings represent meaning."

Lexical Search (BM25)

Precision for exact phrases, codes, and SKUs.

+

Dense Retrieval

Recall for synonyms, intent, and concept matching.

=

Hybrid Search

Robust retrieval across all query types.

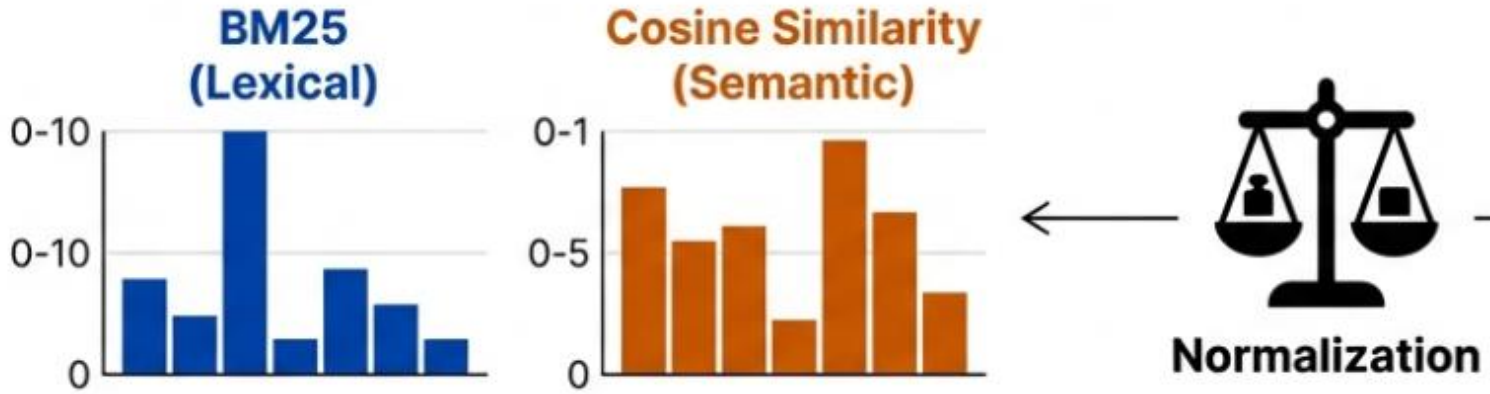
Deploying dense retrieval alongside lexical methods ensures coverage for both semantic intent and precise keyword requirements.

Score Fusion

0.3 BM25 0.7

$$S(d) = \alpha \cdot \text{LexicalScore} + (1 - \alpha) \cdot \text{SemanticScore}$$

Controls balance



CRITICAL: Must normalize scores first (e.g., Min-Max or Z-score) because BM25 and Cosine Similarity operate on different scales.

Best For: When you can keep score scales stable and tuned.

Reciprocal Rank Fusion (RRF)

$$\frac{1}{1} \quad \frac{1}{3} \quad 0.\overline{33} \quad 3/2 \quad \frac{1}{1} + \frac{1}{2} \quad 2 \quad +60$$

$$\mathrm{RRF}(d) = \frac{1}{k + \mathrm{Rank}_{\mathrm{Lexical}}} + \frac{1}{k + \mathrm{Rank}_{\mathrm{Semantic}}}$$

- Uses position (rank), not score value.
- k is a constant (Commonly 60 for production, 10 for teaching).
- **Logic:** If a doc is high in either list, it scores well. If high in both, it scores very well.

Best For: When you don't want to worry about normalizing score scales.

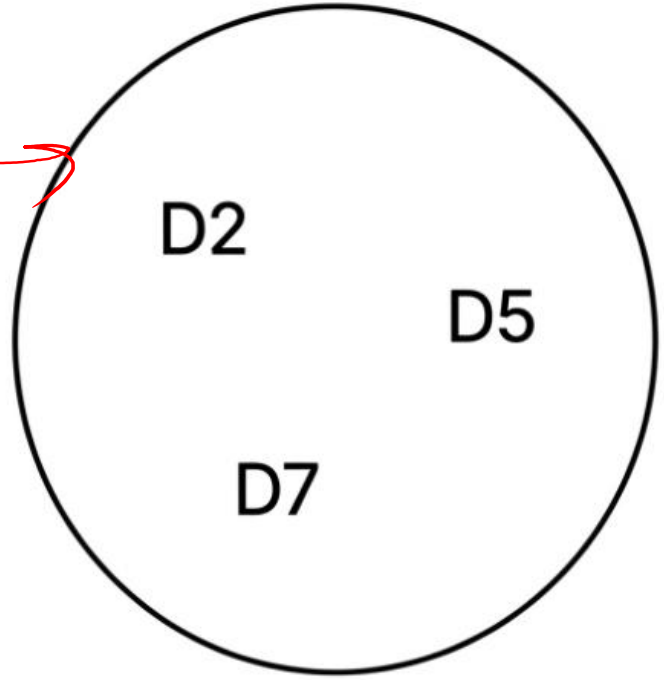
Evaluating Retrieval

S → Adv → Reset

P R

Ground Truth (Relevant Set)

q



|R| = 3 (Total Relevant Items)

Ranked Results (System Output)

1. D5 ✓
2. D1 ✗
3. D2 ✓
4. D9 ✗
5. D7 ✓

Precision: How well did we retrieve?

Ranked Results (System Output)

1. D5 ✓
2. D1 ✗
3. D2 ✓
4. D9 ✗
5. D7 ✓

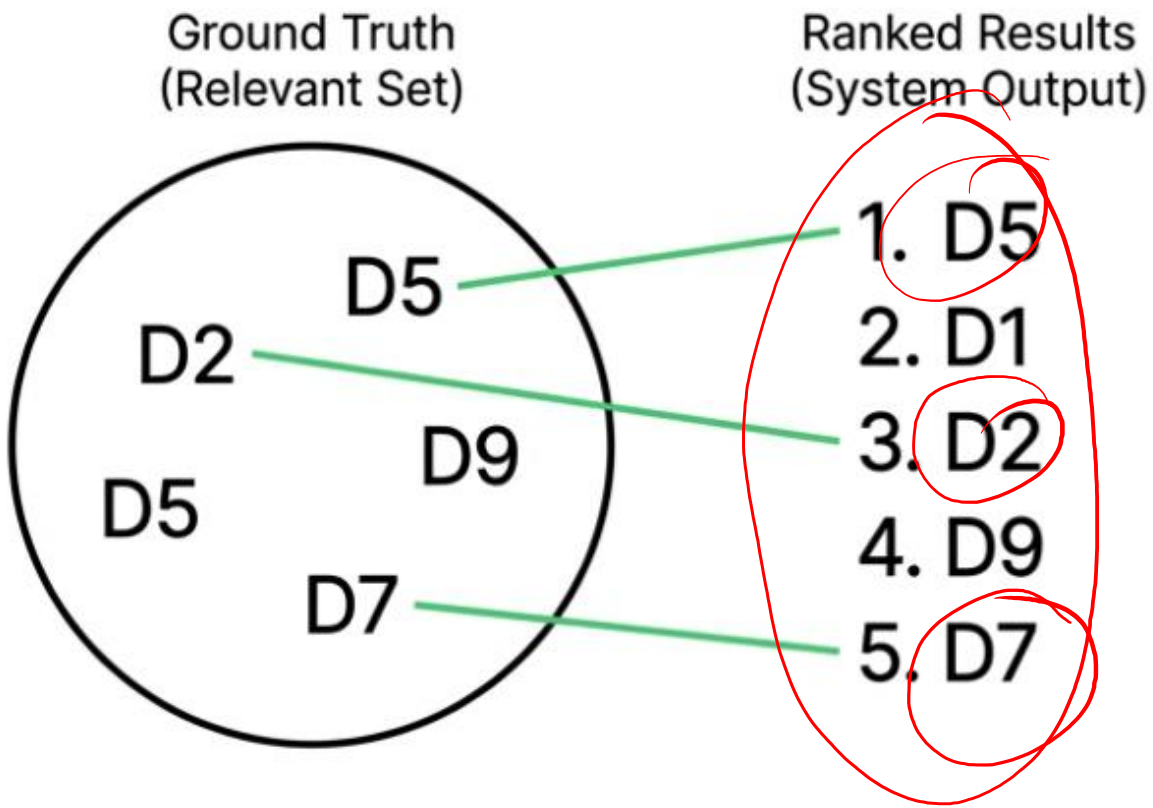
Total Retrieved = 5

Precision = Relevant Retrieved / Total Retrieved

P = 3 / 5 = 0.60

Penalizes noise (False Positives). Critical when wrong answers are costly, such as in medical policy or compliance search.

Recall: Did we miss anything?



$$\text{Recall} = \frac{\text{Relevant Retrieved}}{\text{Total Relevant in Corpus}}$$

$$R = 3 / 3 = 1.00$$

Penalizes omission (False Negatives). Critical when missing data is dangerous, such as e-discovery, safety documentation, or forensic audits.

@K

P@1 P@3 P@5 P@10

Ranked Results (System Output)

1. D5 ✓
2. D1 ✗
3. D2 ✓

The Cutoff (K=3)

4. D9 ✗
5. D7 20%

Precision@3

2 relevant / 3 shown = 0.67

Recall@3

2 retrieved / 3 exist = 0.67

@5 @10

Metrics must reflect the user's viewport. Vital for mobile UIs or RAG systems passing limited context windows to an LLM.

Average Recall / Precision

1. ✓ D5 → P@1 = 1/1 = 1.00
2. ✗ D1
3. ✓ D2 → P@3 = 2/3 = 0.67
4. ✗ D9
5. ✓ D7 → P@5 = 3/5 = 0.60

Simply finding the right documents isn't enough. AP rewards systems that place the right answers *earlier* in the list.

AP = (1.00 + 0.67 + 0.60) / 3 = 0.75

Reciprocal Rank

1. **D5**
2. D1
3. D2
4. D9
5. D7

$$RR = \frac{1}{\text{Rank of first relevant}}$$

First hit is at Rank 1

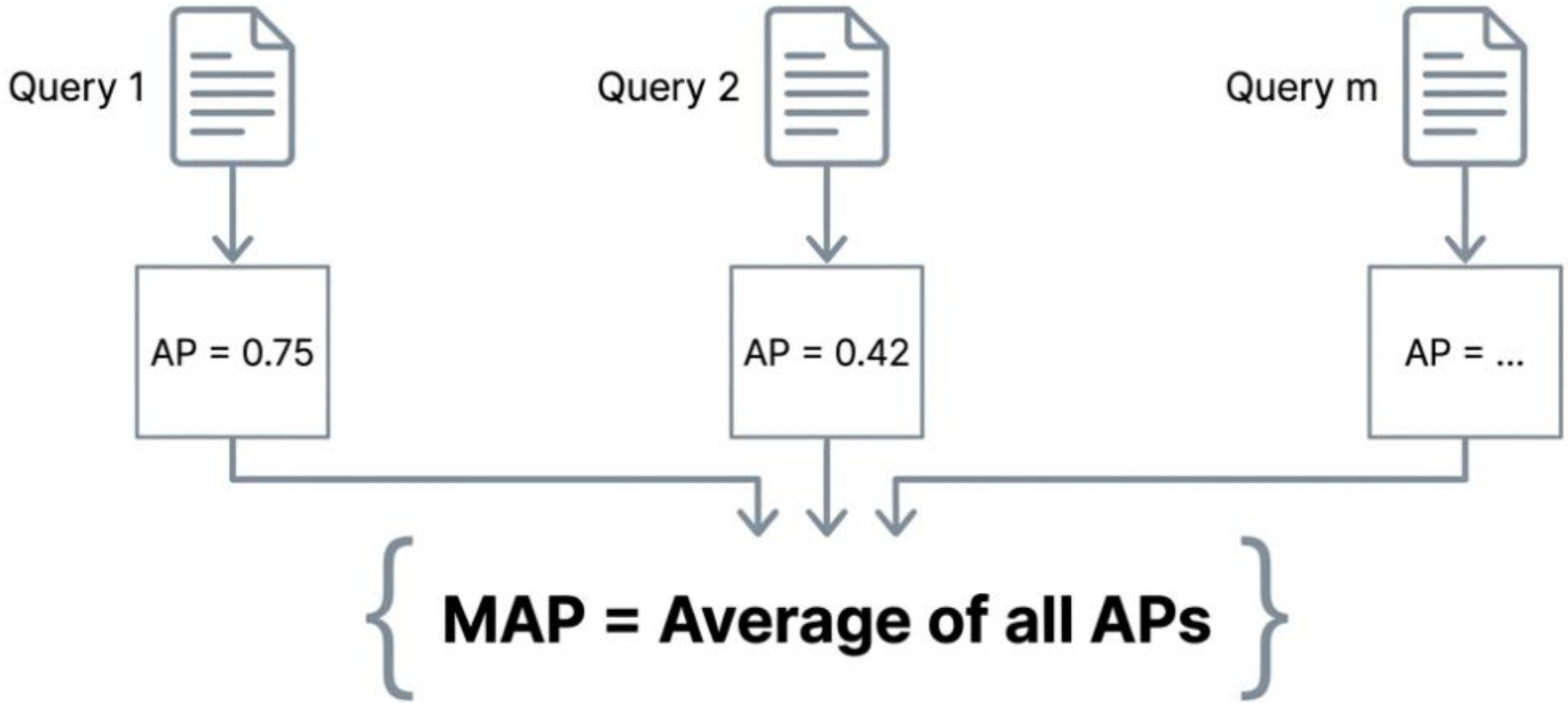
RR = 1 / 1 = 1.0

Used for “Known Item” search, chatbots, or QA. If the user only needs one answer, the rest of the list is irrelevant.

Thank You

Appendix

Mean Average Precision



MAP = (1/m) * Sum(AP)

The gold standard for general search quality. It measures the average performance across all query streams.